

總覽：depot 留得住檔案、答不出哪一版跑的；十六個問題歸成六個原則

這份文件回答：把版控當備份的團隊每天怎麼運作、長出哪些問題、這些問題歸成哪幾個原則



今天這些都由某個人的記憶回答

備份做到了：檔案留得住；「哪一版跑的、能不能重跑、誰看過」答不出。十六個問題歸成六個原則，之後的對策一一對應。

右邊每個框是一個原則，框裡是沒做到它時長出的問題，圖號是頁。Code review 不是原則：要不要由各目錄講好，歸 Self-documenting。

日常：改動在個人 workspace 累積，depot 隔很久才收到一大包

這份文件回答：把版控當備份的團隊，每天具體怎麼做事；問題從這些做法的哪裡長出來 ↓

個人 workspace

p4 client，只有自己看得到



submit 一大包

Perforce depot

大家看得到的那份



這段期間 depot 沒有任何變化

什麼時候才 submit

里程碑到了

有人跟你要

休假前

怕 workspace 壞掉

主管提醒

一包裡面有什麼

好幾個 block 的 RTL

順手改的 script

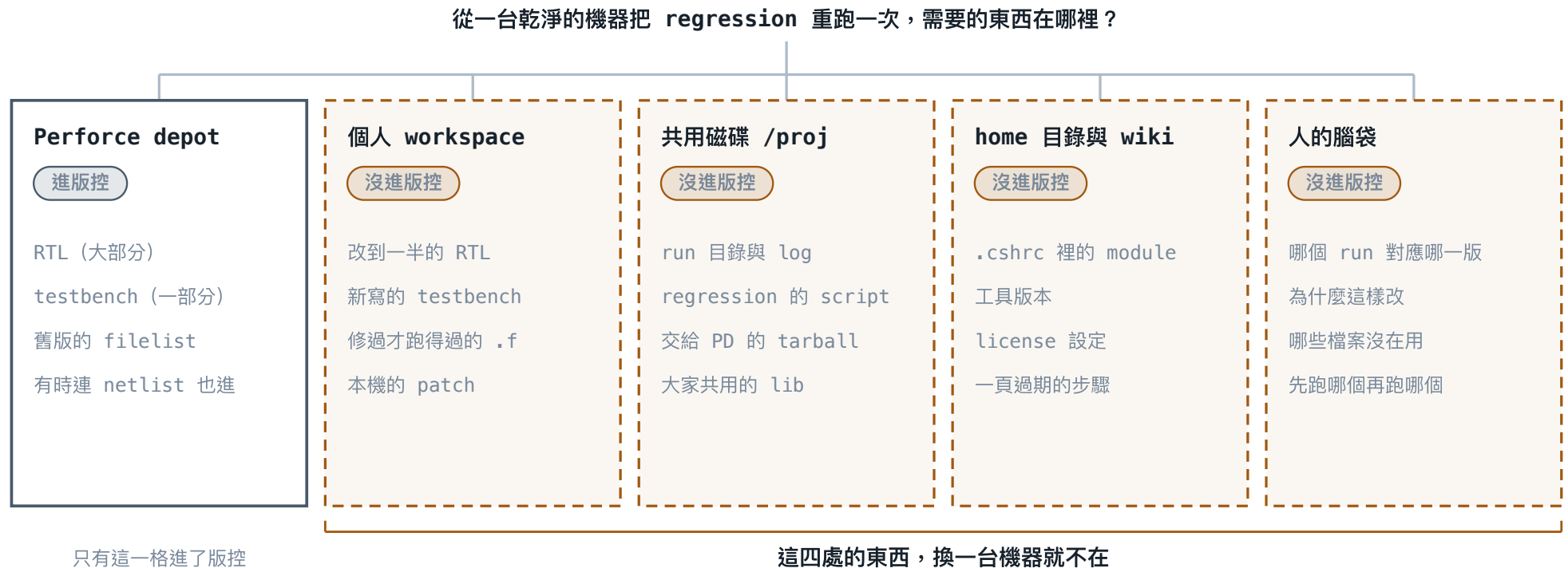
新的 testbench

一起改的 .f

submit 的目的是把檔案存起來，所以在怕丟、有人要、里程碑到了的時候才做。

兩次 submit 之間的所有改動，只存在這個人的 workspace 裡，別人看不到也用不到。

散落：跑 regression 要的東西分在五個地方，depot 只是其中之一



depot 裡只有檔案的一部分；要跑得起來，其餘的東西散在磁碟、**home** 目錄和人的腦袋裡。

換一台乾淨的機器，照 **depot** 的內容做不出同一個結果。

交付：結果靠 email 貼路徑，label 記得檔案版本，記不得工具與環境

一次 regression 的結果怎麼傳出去



里程碑時打一個 label

label RTL_FREEZE_v2

它記得

- ✓ 哪些檔案、各是哪個版本

它不記得

- x simulator 與 synthesis 工具的版本
- x 環境變數與 license 設定
- x 跑了哪些測試、結果如何
- x 怎麼跑：script 在誰的目錄

結果和它的來源之間沒有連結：email 裡的路徑指向一個會變的目錄，label 只記得檔案。

想知道某個結果是怎麼來的，只能找到跑的那個人問。

問題：「這份結果是哪一版跑的」 要問好幾個人，答案仍是大概



沒有紀錄的地方

改動只在 workspace，沒有 CL 號碼

label 沒記工具版本

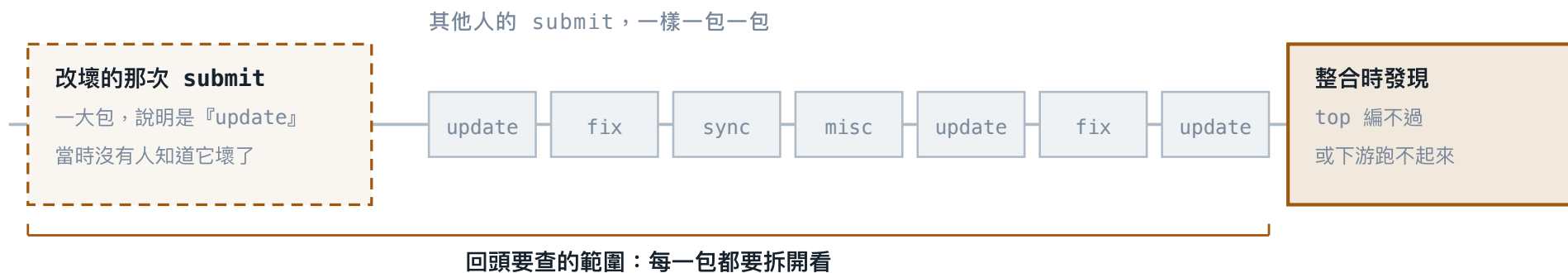
run 目錄會被覆蓋

跑法在個人的 script 裡

沒有任何一筆紀錄把結果、檔案版本、工具版本和跑法綁在一起。

答案由幾個人的記憶拼出來，每一段都帶著「應該」。

問題：壞掉被發現時，離改壞它的那次 submit 已經很遠



怎麼找

- 一包一包拆開，猜哪幾個檔案有關
- 逐個問當時改了什麼
- 在自己的 workspace 把舊版換回來試

誰先發現

整合的人 PD DV 韌體 客戶

發現的人離那次改動最遠，手上沒有線索

改動進 depot 時沒有任何檢查，問題要等到整合或下游才浮現。

從發現點往回找，中間每一包都要拆開看；一次送進的東西越多，越難切。

問題：下游收到的包沒有清單；流程只在人腦裡，人走了就斷

交給下游



下游問不出來的

- ✗ 跟上一包差在哪？
- ✗ 哪些是刻意改的、哪些是順手改的？
- ✗ 這包和 label 的內容一樣嗎？
- ✗ 出問題了，要回報哪一版？

人離開的時候



下游收到的是一包檔案，拿不到它的來歷。

工作流程存在人身上，人離開的時候，depot 留不住它。

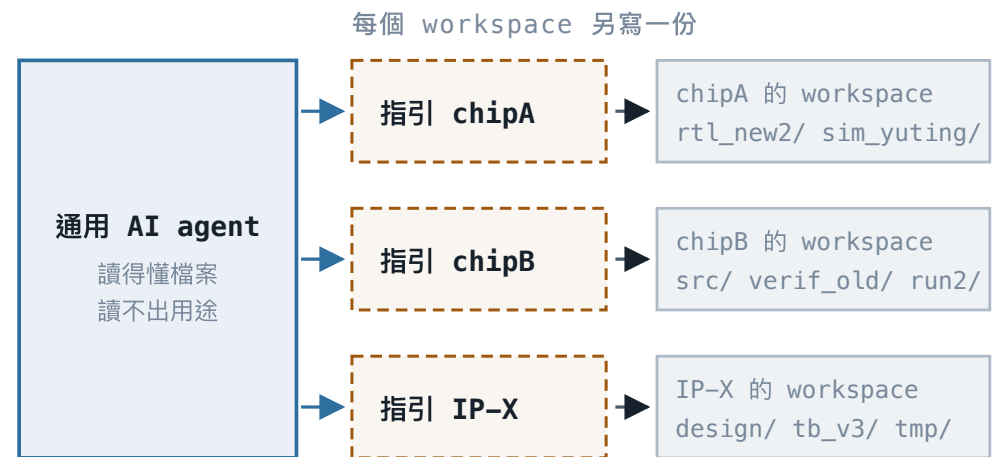
問題：目錄用途沒寫在 depot，新人要人帶，AI agent 也要人另寫說明

第一次打開專案目錄看到的：用途與相依都要猜

/proj/chipA/	
rtl/	? 哪些檔案還在用?
rtl_old/	? 能刪嗎?
rtl_new2/	? 跟 rtl/ 差在哪?
sim/	
sim_yuting/	? 個人的，還是正式的?
scripts/	? 哪個才是正式的?
scripts_bak/	
release_0917/	? 跟 label 的內容一樣嗎?
tmp/	
top.f	? 引用的檔案一半在 rtl_new2/
README	? 寫的是上一個專案

新來的工程師：問同事、慢慢摸；摸出來的地圖留在他腦袋裡

通用的 AI agent 進到這些 workspace



指引誰寫得出來？

只有知道的那幾個人

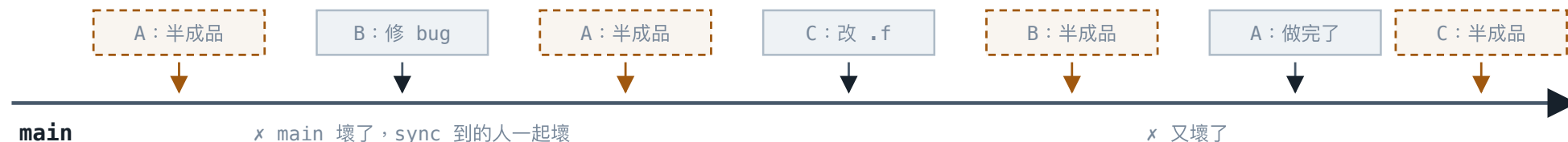
有幾個 workspace 就要寫幾份

目錄的用途與相依關係沒有寫下來，新人靠問、靠摸；地圖最後還是只在人的腦袋裡。

通用的 AI agent 一樣看不懂，每個 workspace 都得由知道的人另寫一份指引。

問題：沒有開發 branch，半成品留在 workspace 或進 main，main 隨時會壞

今天：只有一條 main，所有人直接往上 submit



少了的那條線：一段工作做完、驗過，再併回 main



沒有這條線的時候

- x 半成品留在 workspace，隔很久才進一大包（圖 1）
- x 半成品直接進 main，sync 到的人一起壞
- x 沒有「做完、驗過」的關卡，CI 沒有觸發點
- x 一段工作和別人的改動混在 main 上，撤不回來
- x 第二個版本靠複製目錄：rtl_new2/（圖 7）

branch 給半成品一個可以常常 submit、又不會弄壞 main 的地方；少了它，Small batches 和 CI 都沒有地方發生。

複製目錄成了 branch 的代替品，repo 因此越來越看不懂。

問題：沒講好哪些目錄要 review，預設沒人看，改 interface 那包也一樣

今天：submit 就是完成



0 個人在 submit 之前
看過這一包

沒有任何目錄講過要不要 review；預設就是沒人看

review 只發生在里程碑的 design review 會議，看投影片與架構
會議上看不到任何一個 CL；連改 interface 的那一包也沒人逐包看

危害

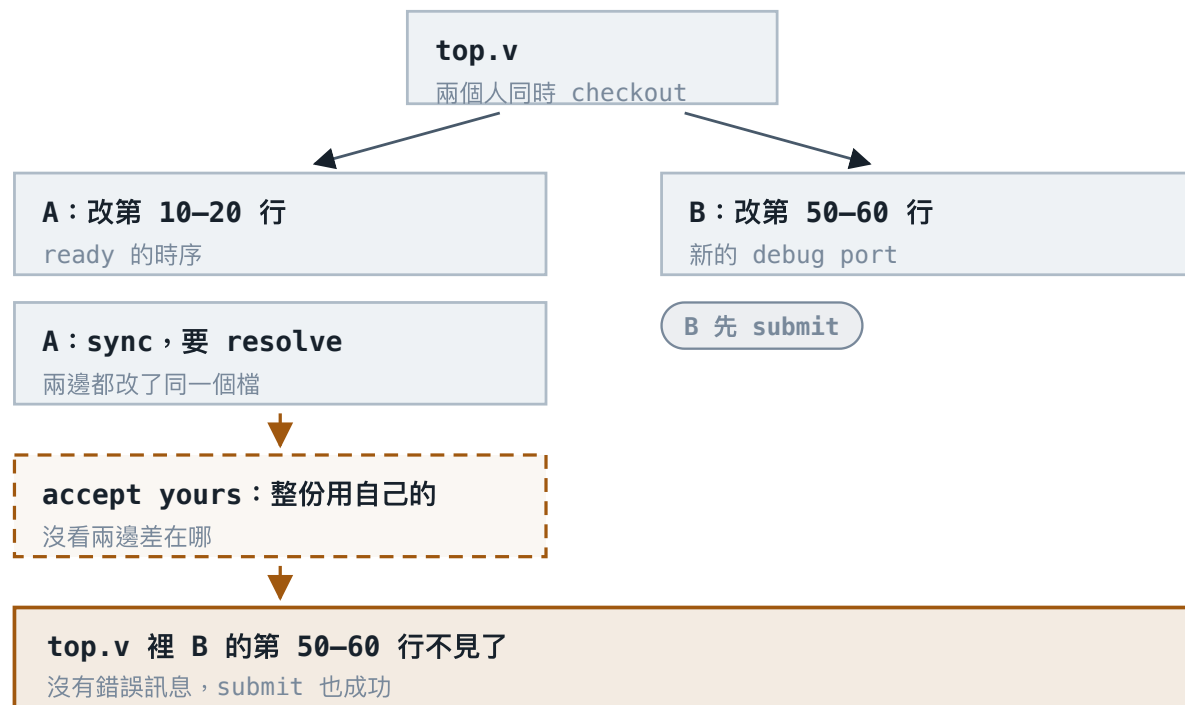
- ✗ interface 改了，用它的人整合時才知道
- ✗ 錯誤要等機器或下游來抓
- ✗ 為什麼這樣改，只有作者知道
- ✗ 新人的寫法沒人修正，變成習慣

示意：A 改了 bus 的 ready 行為，B 的 block 到整合才發現

要不要 review 由各目錄自己定、可以改，但要講好、寫下來；沒講好，預設就是沒人看。

連改 interface 的那一包也一樣；design review 的會議看不到任何一個 CL。

問題：兩人改同一個檔，**resolve** 整份收下，另一人的改動消失



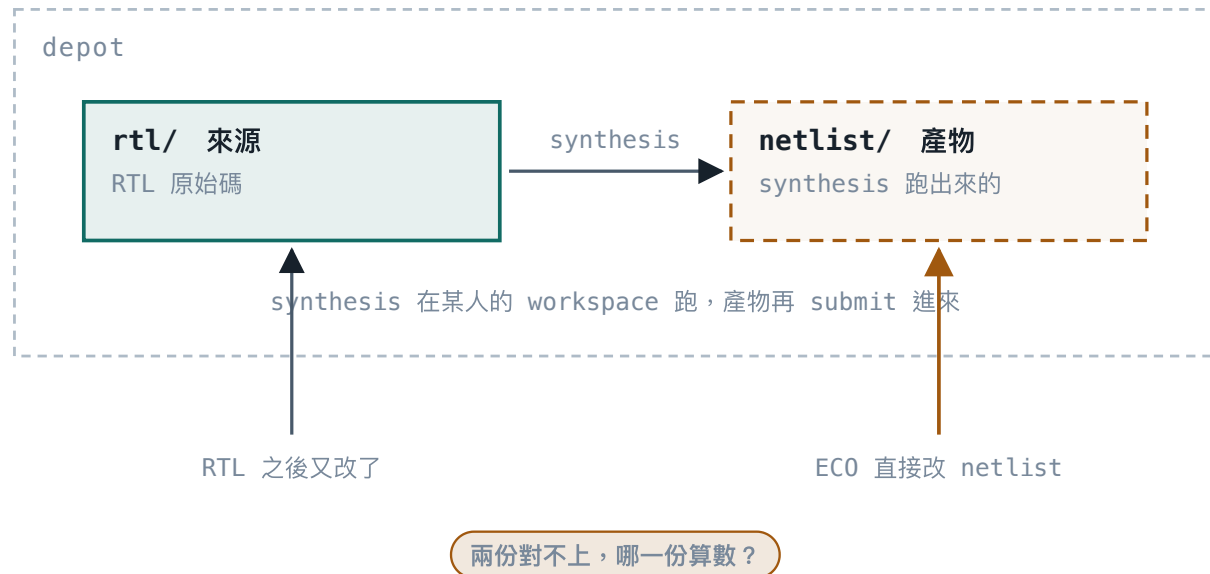
危害

- ✗ 改動消失，沒有任何錯誤訊息
- ✗ 要等下游壞了才發現，然後互相指責
- ✗ 為了躲開，改成鎖檔或各改各的複本
- ✗ 複本越多，離只有一份真相越遠

resolve 需要看懂兩邊的改動；沒人看，就整份收下自己的。

版控給的是合併的機制，習慣上當成覆蓋。

問題：netlist 等產物和來源一起進 depot，改哪一份才算數沒人說得清



危害

- ✗ 兩份真相，改哪一份才算數沒人說得清
- ✗ ECO 改在產物上，重新產生一次就消失
- ✗ 產物對應哪一版來源，沒有紀錄
- ✗ depot 越來越大，sync 越來越久

下游拿的是 netlist；再跑一次 synthesis，ECO 就被蓋掉

產物該由來源產生；進了 depot，它就成了第二份真相。

改產物比改來源快，所以 ECO 都改在產物上；下一次重新產生，就把它蓋掉。

問題：第三方 IP 解壓覆蓋，晶片裡是哪一版沒人說得出

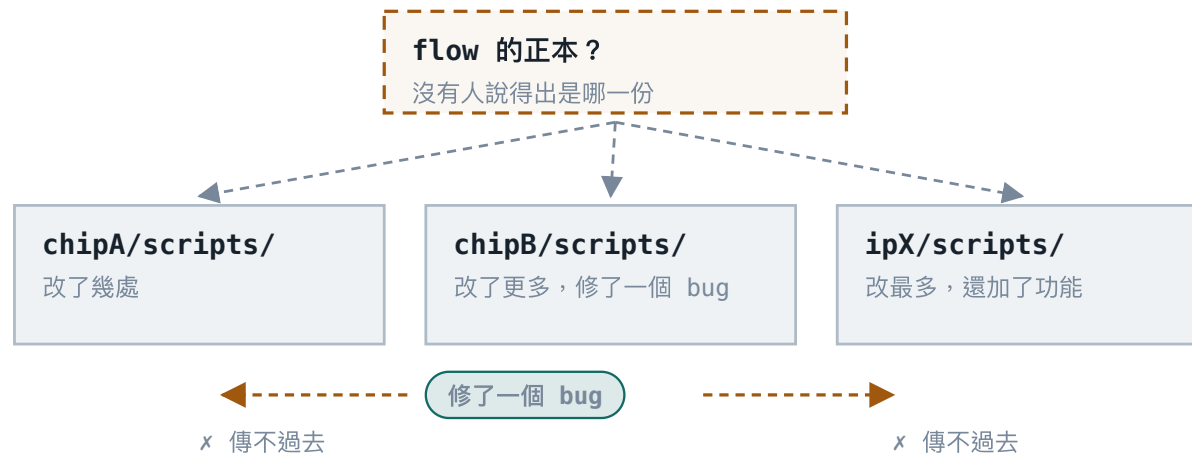


示意：lint patch 在 v1.3 解壓時被蓋掉，重新 lint 才發現

IP 以 tarball 進來、解壓覆蓋，版控裡看不出 drop 與 drop 之間的差異。

每一次 drop 都是一次沒有紀錄的大包 submit。

問題：flow script 每個專案複製一份改，修好的 bug 傳不出去



危害

- x 同一個 bug 在每個專案各修一次
- x 新專案不知道該從哪一份複製
- x 各專案 flow 行為不同，結果不能比
- x 沒有人知道哪一份是正本

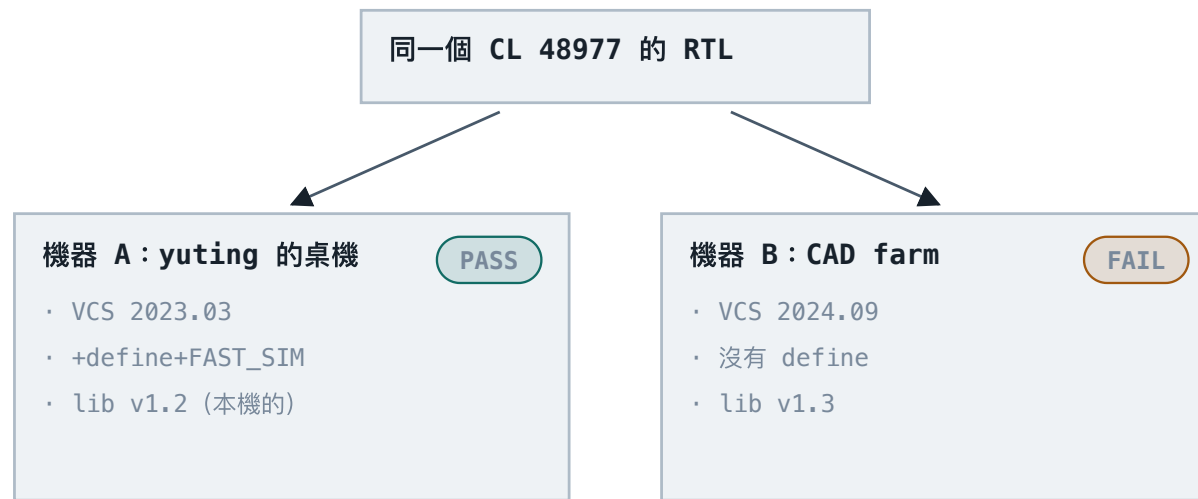
複製的時候很快，複製之後各自演化

同一個 bug 在 chipA 與 ipX 還在；新專案不知道該從哪一份複製

複製是最快的共用方式，複製之後就各自演化。

修好的東西留在修的那個專案裡。

問題：同一份 RTL 兩台機器跑出不同結果，分不出哪個才對



危害

- ✗ PASS 不代表對，FAIL 不代表錯
- ✗ 報告上的 PASS 不知道是哪個環境跑的
- ✗ 搬到 farm 或新機器就壞
- ✗ 找差異只能逐項比 .cshrc

差在哪？沒有任何紀錄：`.cshrc`、`alias`、環境變數、`license feature`

兩個人各自相信自己的環境，報告上只寫 PASS

跑的環境沒有跟著 RTL 一起進版控，同一份來源在不同機器上就是不同的設計。

結果能不能信，取決於跑的人記不記得自己的設定。

問題：regression 狀態靠人填 Excel，表和實際結果對不上



危害

- ✗ 表是上週的，結果是今天的
- ✗ PASS 的定義每個人不同
- ✗ 版本欄靠回想，對不回 CL
- ✗ 沒人敢信，最後還是去問跑的人

status.xlsx (示意)		
test	result	version
t_dma_basic	PASS	
t_dma_burst	PASS	上週五的
t_irq_all	?	

← log 其實還沒跑完

← 版本靠回想

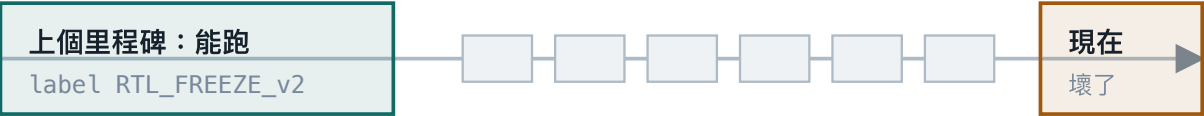
← 沒人填

表是機器結果的手抄本，抄的那一刻就開始脫節

結果由機器產生，狀態卻由人抄寫；抄的時候就跟實際脫節。

狀態表回答不了「這是哪一版、哪個環境跑的」。

問題：想退回上次能跑的狀態，檔案回得去，環境回不去



想退回去

- | | |
|------------------|--------------------|
| ✓ sync 到 label | 檔案回去了 |
| x 工具版本 | 已經換了，舊版 license 沒了 |
| x /proj 的 script | 已經被改過 |
| x ip/ | 被下一個 drop 蓋掉 |
| x 環境 | .cshrc 改過，沒人記得舊的 |

還是壞的
退不回去，只能往前修

危害

- x label 只能回檔案，回不了環境與產物
- x 退不回去，只能往前硬修
- x freeze 靠複製目錄，因為回不去
- x 風險只能累積，不能歸零

所以 tape-out 前的 freeze 靠複製整個目錄：因為知道回不去

回到一個能跑的狀態需要檔案、工具、環境、IP 一起回去；版控只記了檔案。
所以沒有人退回去，大家只往前修。

總結：備份做到了，「哪一版跑的、能不能重跑」一個都答不出

depot 做到的

- ✓ 檔案的最新版在哪
- ✓ 誰在什麼時候改過哪個檔案
- ✓ 舊版救得回來
- ✓ 里程碑那天的檔案清單 (label)

備份要做的事，它都做到了

depot 答不出來的

- x 這個結果是哪一版做的？
- x 從乾淨的機器能重現嗎？
- x 哪一次改動弄壞了它？
- x 下游收到的跟上一版差在哪？
- x 跑的步驟與環境在哪裡？
- x 這個檔案還有人在用嗎？
- x 這個目錄裝什麼、靠哪些東西？
- x main 現在能用嗎？
- x 這一包併入前誰看過？
- x 晶片裡是哪一版 IP？

今天誰在回答

- 跑的人
- 沒人試過
- 大家一起猜
- 上游工程師
- CAD 加跑的人
- 最資深的人
- 帶你的那個人
- sync 了才知道
- 沒有人
- 解壓的那個人

備份的功能它確實做到了；右邊每一題，今天都由某個人的記憶回答。

AI 的產出要跨團隊被採用，先得讓紀錄能回答右邊這些題。

收斂：前面的問題歸成 SSOT、CI、CD 等六個原則，對策照原則一一對應

原則	意思，以及做得到 / 做不到的檢驗	沒做到時的問題（圖號）
Small batches 小步常進	改動小而頻繁地進到共用的地方，每一包說得出改了哪一件事 檢驗：隨便挑一包 submit，說得出它改了哪一件事	圖 2 圖 6 圖 9 圖 11
Single Source of Truth SSOT 單一事實來源	跑得起來需要的一切都在版控裡，而且只有一份；產物由來源產生 檢驗：換一台乾淨的機器，只靠版控的內容做出同一個結果	圖 3 圖 4 圖 7 圖 12 圖 13 圖 14 圖 15
Traceability 可追溯	每個結果與交付物都連得回產生它的版本、工具、環境與步驟 檢驗：隨便拿一份結果，說得出它的版本、工具、環境與步驟	圖 4 圖 5 圖 7 圖 15 圖 16 圖 17
Continuous Integration CI 變更即驗證	改動進來的當下就被機器檢查，結果由機器寫下；共用的 main 隨時可用 檢驗：改壞的那一包進來時就被標出來，用不著等到整合	圖 6 圖 9 圖 16
Self-documenting 自我描述	目錄的用途、相依、怎麼跑、要不要 review，寫在 repo 裡 檢驗：第一次來的人只讀 repo，就說得出每個目錄的用途、相依、要不要 review	圖 7 圖 8 圖 10 圖 14
Continuous Delivery CD 持續交付	過 check 就自動打包附 manifest 放到取用處；release 按一下就出 檢驗：任何時候不用問人就拿得到最新一份附 manifest 的交付包，下游拿了就能跑	圖 4 圖 7 圖 17

六個原則都是 repo 該有的性質，各有一個做得到或做不到的檢驗。

之後談對策，每一條只回答一個問題：它讓哪一個檢驗從做不到變成做得到。

Code review 不是原則：要不要 review 由各目錄自己定、可以改，但要講好、寫在 PROJECT_MAP——這歸 Self-documenting。